Dev Guide 用户文档

PingCAP Inc.

20220402

Table of Contents

1 创建和管理数据库对象 **6**

1.1 创建和管理库 6

1.1.1 创建和管理库 6

1.1.2 库命名规范 6

1.2 创建和管理表 6

1.2.1 创建和管理表 6

1.2.2 表命名规范 6

1.2.3 表设计 7

1.3 创建和管理索引 7

1.3.1 创建和管理索引 7

1.3.2 索引命名规范 7

1.3.3 索引设计 8

2 TiDB 事务说明 **8**

2.1 TiDB 事务概览及模型等其他内容 8

3 错误处理及故障排查 **8**

3.1 故障诊断 8

3.1.1 各类故障诊断 8

3.2 使用 TiFlash 8

3.2.1 TiFlash 常见问题 9

4 性能优化及建议 **9**
4.1 系统调优 9
4.2 软件调优 9
4.3 SQL 性能调优 9

5 数据迁移 **9**
5.1 各类数据迁移 9

6 常见参数说明 **9**
6.1 命令行参数 9
6.2 配置文件参数 10

7 SQL 开发规范及基本原则 **10**
7.1 对象命名规范 10
7.2 数据库命名规范 10
7.3 表命名规范 10
7.4 字段命名规范 10
7.5 索引命名规范 11
7.6 表的设计 11
7.7 字符集 11
7.8 列的自增属性 11
7.9 创建、删除表规范 12
7.10 变更表规范 12
7.11 大事务处理 12
7.12 SELECT * 使用规范 12
7.13 防范写入热点创建规范 12
7.14 数据删除规范 13
7.15 分页查询 order by 语法使用规范 13
7.16 group by 语法使用规范 13
7.17 模糊匹配 like 语法使用规范 13
7.18 视图使用规范 13
7.19 多表关联查询规范 14
7.20 分区表规范 14

8 TiDB 数据库开发规范 14

8.1 前言 14

8.1.1 1. 目的 14

8.1.2 2. 适用范围 14

8.1.3 3. 注意事项 15

8.2 对象命名规范 15

8.2.1 1. 原则 15

8.2.2 2. 数据库命名规范 15

8.2.3 3. 表命名规范 16

8.2.4 4 字段命名规范 16

8.2.5 5 索引命名规范 16

8.3 数据库对象设计 17

8.3.1 1. 表的设计 17

8.3.2 2 字段的设计 17

8.3.3 3. 字段默认值 19

8.3.4 4. 索引的设计 20

8.3.5 5. 权限的设计 21

8.3.6 6. 约束的设计 21

8.3.7 7. 分区表的设计 22

8.4 数据模型设计 22

8.4.1 1. 完整性 22

8.4.2 2. 性能 23

8.4.3 3. 扩展性 23

8.5 SQL 开发规范 23

8.5.1 1. 建表删表规范 23

8.5.2 2. select * 使用规范 24

8.5.3 3. 大事务处理 24

8.5.4 4. 字段上使用函数规范 24

8.5.5 5. 数据删除规范 25

8.5.6 6. 其他规范 25

8.6 事务限制 25
8.6.1 1. 隔离级别 25
8.6.2 2. SI 可以克服幻读 26
8.6.3 3. SI 不能克服写偏斜 26
8.6.4 4. 不支持 savepoint 和嵌套事务 27
8.6.5 5. 大事务限制 27
8.6.6 6. 自动提交的 SELECT FOR UPDATE 语句不会等锁 28
8.7 隐式类型转换 28
8.7.1 1. 转换规则 28
8.7.2 2. 隐式类型转换引起的后果 29
8.8 结果集不稳定 30
8.8.1 1. group by 30
8.8.2 2. order by 32
8.8.3 3. 由于 group_concat() 中没有使用 order by 导致结果集不稳定 33
8.8.4 4. select * from t limit n 的结果不稳定 35
8.9 索引的使用注意 35
8.9.1 1. TiDB 中的索引 36
8.9.2 2. 复合索引的设计 36
8.10 自增列的使用注意 37
8.10.1 1. 原理 37
8.10.2 2. 最佳实践 37
8.10.3 3. 显式赋值的后果及挽救措施 37
8.11 TiDB 中的各种超时 38
8.11.1 1. GC 超时 38
8.11.2 2. 事务超时 38
8.11.3 3. SQL 执行时间超时 38
8.11.4 4. JDBC 查询超时 39
8.12 JDBC 最佳实践 39
8.12.1 1. MySQL Connector/J 推荐版本 39
8.12.2 2. JDBC 参数设置 39

8.13 缓解热点问题 40
8.13.1 1. 写入热点 40
8.13.2 2. 热点的观测和定位 40
8.13.3 3. 写入热点的缓解 41
8.13.4 4. 读取热点 42
8.14 分页的最佳实践 43
8.14.1 1. 分页查询的基本原则 43
8.14.2 2. 单字段主键表的分页批处理 44
8.14.3 3. 复合主键表的分页批处理 45
8.15 唯一序列号生成方案 46
8.15.1 1. 自增列 46
8.15.2 2. 序列 (Sequence) 47
8.15.3 3. 类 Snowflake 方案 47
8.15.4 4. 号段分配方案 47
8.16 流程规范 48

1 创建和管理数据库对象

1.1 创建和管理库

本文介绍创建和管理数据库的基本 SQL 操作和规范。

1.1.1 创建和管理库

SQL 语句参阅：

- CREATE DATABASE
- DROP DATABASE
- ALTER DATABASE

1.1.2 库命名规范

建议按照业务、产品线或者其它指标进行区分，一般不要超过 20 个字符。如：临时库（tmp_crm）、测试库（test_crm）。

1.2 创建和管理表

本文介绍创建和管理数据表的基本 SQL 操作和规范。

1.2.1 创建和管理表

SQL 语句参阅：

- CREATE TABLE
- DROP TABLE
- FLASHBACK TABLE
- RENAME TABLE
- ALTER TABLE
- RECOVER TABLE

1.2.2 表命名规范

1. 同一业务或者模块的表尽可能使用相同的前缀，表名称尽可能表达含义；
2. 多个单词以下划线分隔，不推荐超过 32 个字符。
3. 建议对表的用途进行注释说明，以便于统一认识，例如：
    - 临时表（`tmp_t_crm_relation_0425`）。

- 备份表（bak_t_crm_relation_20170425）。
- 业务运营临时统计表（tmp_st_[业务代码]_[创建人缩写]_[日期]）。
- 账期归档表（t_crm_ec_record_YYYY[MM][DD]）。

4. 不同业务模块的表单独建立 DATABASE，并增加相应注释。

1.2.3 表设计

1. TiDB 中的一张表的行（Rows）是按照主键的字节序排序的（整数类型的主键我们会使用特定的编码使其字节序和按大小排序一致），即使在 CREATE TABLE 语句中不显式的创建主键，TiDB 也会为该表分配一个隐式主键

2. 在进行 TiDB 表设计过程中，有以下几点需要注意：

   - 表需要有主键或者唯一索引。
   - 尽量选择有意义的列作为主键。
   - 出于为性能考虑，尽量避免存储超宽表，表字段数不建议超过 60 个，建议单行的总数据大小不要超过 64K，数据长度过大字段最好拆到另外的表。
   - 不推荐使用复杂的数据类型。
   - 需要 JOIN 的字段，数据类型保障绝对一致，避免隐式转换。

1.3 创建和管理索引

本文介绍创建和管理索引的基本 SQL 操作和规范。

1.3.1 创建和管理索引

SQL 语句参阅：

- CREATE INDEX
- ADD INDEX
- DROP INDEX
- RENAME INDEX
- SHOW INDEXES [FROM|IN]

1.3.2 索引命名规范

- 唯一索引：uk_[表名称简写]_[字段名简写]
- 普通索引：idx_[表名称简写]_[字段名简写]
- 多单词组成的 column_name，取尽可能代表意义的缩写。

1.3.3 索引设计

- 选择区分度大的列建立索引，不在低基数列上建立索引，例如：“性别”，“是否是 XXX”；
- 单张表的索引数量控制在 5 个以内，避免冗余索引；
- 索引中的字段数建议不超过 5 个；
- 唯一索引建议由 3 个以下字段组成；
- 尽量不要在频繁更新的列上创建索引；
- 对于确定需要组成组合索引的多个字段，建议将选择性高的字段靠前放；
- 最左前缀原则，使用联合索引时，从左向右匹配，比如索引 `idx_c1_c2_c3 (c1,c2,c3)`，相当于创建了 (c1)、(c1,c2)、(c1,c2,c3) 三个索引，`WHERE` 条件包含上面三种情况的字段比较则可以用到索引，但像 `WHERE c1=a and c3=c` 只能用到 c1 列的索引，像 `c2=b and c3=c` 等情况就完全用不到这个索引；
- 不对过长的 `VARCHAR` 字段建立索引；
- 定期删除一些长时间未使用过的索引；
- `ORDER BY`，`GROUP BY`，`DISTINCT` 的字段需要添加在索引的后面，形成覆盖索引；
- 新的 `SELECT`，`UPDATE`，`DELETE` 上线，都要先 `EXPLAIN`，确保索引的正确性；
- 不建议在 `WHERE` 条件索引列上使用函数，会导致索引失效，如 `lower(email)`；
- 使用 `LIKE` 模糊匹配，`%` 不要放首位，会导致索引失效。

2 TiDB 事务说明

本文对 TiDB 事务原理及事务模型进行说明。

2.1 TiDB 事务概览及模型等其他内容

参阅 TiDB 事务概览及模型等其他内容

3 错误处理及故障排查

3.1 故障诊断

本文对 TiDB 集群在使用中遇到的常见问题及故障提供诊断及处理说明。

3.1.1 各类故障诊断

参阅 TiDB 集群故障诊断常见问题及其他内容。

3.2 使用 TiFlash

本文对 TiFlash 在使用中的常见问题进行解析，并提供处理说明。

3.2.1 TiFlash 常见问题

参阅TiFlash 常见问题。

4 性能优化及建议

本文介绍 TiDB 集群在使用中的各类调优手段，包括系统调优、软件调优、SQL 性能调优。

4.1 系统调优

参阅操作系统性能参数调优。

4.2 软件调优

参阅 TiDB 内存调优及其他内容。

4.3 SQL 性能调优

参阅 SQL 性能调优概览及其他内容。

5 数据迁移

本文档介绍支持从哪些路径将数据迁移到 TiDB，包括从 MySQL 迁移到 TiDB 和从 CSV/SQL 文件迁移到 TiDB。

5.1 各类数据迁移

参阅数据迁移概述及各类迁移内容。

6 常见参数说明

本文介绍 TiDB 各组件命令行参数及配置文件参数，配置文件参数及比命令行参数支持更多的选项。

6.1 命令行参数

参阅 TiDB 配置参数及其他内容。

6.2 配置文件参数

参阅 TiDB 配置文件描述及其他内容。

7 SQL 开发规范及基本原则

本文主要介绍在开发业务及应用时，需要遵守的 SQL 规范及基本原则。

7.1 对象命名规范

- 命名建议使用具有意义的英文词汇，词汇中间以下划线分隔；
- 命名只能使用英文字母、数字、下划线；
- 避免用 TiDB 的关键字，如 group，error，rank 等作为对象名；
- 建议所有的数据库对象使用小写字母；
- 所有的数据库对象的命名请注意标识符长度硬限制 ；

7.2 数据库命名规范

- 按照业务模块、产品线和访问权限隔离需求来创建数据库；
- 建议数据库名称不要超过 16 个字符，如：基础信息库 (basicinfo_db)、认证中心库 (certifycenter_db)；

7.3 表命名规范

- 同一业务或者模块的表尽可能使用相同的前缀，表名称尽可能表达含义；
- 多个单词以下划线分隔，不推荐超过 32 个字符；
- 建议对表的用途进行注释说明，以便于统一认识；临时用统计表 (tmp_t_crm_relation_0425) 备份表 (bak_t_crm_relation_20170425)
- 只支持将 lower-case-table-names 值设为 2，即数字字典中记录的表名区分大小写，匹配查找表名时不区分大小写；

7.4 字段命名规范

- 字段命名需要表示其实际含义的英文单词或简写；
- 建议各表之间相同意义的字段应同名，并且一定使用相同的字段类型；
- 字段也尽量添加注释，枚举型需指明主要值的含义，如”0 - 离线，1 - 在线”；
- 布尔值列命名为 [is_ 描述]。如 member 表上表示为 enabled 的会员的列命名为 is_enabled；
- 字段名不建议超过 32 个字符；

7.5 索引命名规范

- 主键索引：pk_[表名称简写]_[字段名简写] ；
- 唯一索引：uk_[表名称简写]_[字段名简写] ；
- 普通索引：idx_[表名称简写]_[字段名简写] ；
- 多单词组成的字段名，使用能代表意义的缩写；

7.6 表的设计

- 表需要有主键或者非空唯一索引，能与各项复制工具更好地兼容；
- 主键不要带有业务含义；
- 业务表使用自增主键时，字段类型推荐使用 bigint unsigned，最大值可达 18446744073709551615；
- 出于为性能考虑，尽量避免存储超宽表，数据长度过大的字段最好拆到独立的表存储，建议表字段数不超过 60 个，建议单行的总数据大小不要超过 64K，TiDB 硬限制单行的总数据大小不超过 6 MB；
- 不推荐使用复杂的数据类型，如 blob 或者 json；
- 进行 join 的关联字段，数据类型保证一致，避免隐式转换；
- 不能以范式作为唯一标准或者指导，在设计过程中，需要从实际需求出发，以性能提升为根本目标来展开设计工作。为了提升性能减少表关联，可以适当保存冗余数据做反范式设计；
- 表对象的设计请注意单个 Table 的限制 ；

7.7 字符集

- 建表时只使用默认的 utf8mb4 编码；
- utf8mb4 的默认排序规则为 utf8mb4_bin (区分大小写)；
- 支持 utf8mb4_general_ci (不区分大小写)，但是需要在集群环境部署时配置 新的排序规则框架；
- 其他的排序规则仅为语法支持，实际不支持；

7.8 列的自增属性

- 列的自增属性仅保证唯一，仅能保证在单个计算节点中自增，不保证多个计算节点中自增，不保证自动分配的值的连续性。业务不应该依赖自增属性的连续性和有序性，如记录的插入顺序排序应按照记录的创建时间。带有自增属性的列出现空洞和跳跃插入的现象是正常现象；
- 不要在语句中显式指定具有自增属性的列的值，由 TiDB 自动分配，否则可能会出现值重复冲突；
- 允许移除列的 AUTO_INCREMENT 属性，但是请谨慎评估，移除该属性后不可恢复；

7.9 创建、删除表规范

- 表的建立在遵循表命名规范前提下，如果业务应用内部封装建表删表语句，需要增加判断逻辑，防止业务流程异常中断。例如：create table if not exists table_name 或者 drop table if exists table_name 语句建议增加 if 判断，避免应用侧由于表的改动造成的异常中断；
- 不支持 create table as select 语法，需要改写为表结构复制 create table like ... 和将数据写入的 insert into select ... 的组合语句；

7.10 变更表规范

- 不支持单条 ALTER TABLE 语句中完成多个操作，不能在单个语句中添加多个列或索引，需要更换成多个单个列或索引的操作；
- 不支持对字段类型的有损修改或修改为超集，请参考ddl-的限制；

7.11 大事务处理

- 单个事务的总大小默认不超过 100 MB，最大支持 10 GB，实际的单个事务大小限制还取决于服务器剩余可用内存的大小，执行事务时 TiDB 进程的内存消耗大约是事务大小的 6 倍以上；
- 由于执行事务时 TiDB 进程的内存消耗大约是事务大小的 6 倍以上，事务设置过大，或者 Batch 过高，可能导致 tidb-server OOM，需要评估好内存容量；
- 为了使性能达到最优，需要对大事务按某个业务维度进行拆分，每 100 ~ 500 行提交一个事务；

7.12 SELECT * 使用规范

- 禁止使用 SELECT * 进行查询。建议按需求选择合适的字段列，杜绝直接 SELECT * 读取全部字段，减少网络带宽消耗，有效利用覆盖索引；

7.13 防范写入热点创建规范

- 对于写入量非常大的表，应当通过应用性能压测等方法在测试环境模拟表的热点情况;
- 通过以下三种手段进行配置，规避表的主键写入热点。避免连续自增主键的设计，建议采用雪花算法生成 UUID 主键；主键是非整数的表或者启用 alt-primarykey=true 配置后创建的所有表，使用 SHARD_ROW_ID_BITS 语法创建基于 rowid 的分片方案，例如：CREATE TABLE t (c int) SHARD_ROW_ID_BITS = 4 或 ALTER TABLE t SHARD_ROW_ID_BITS = 4; 创建按照 Hash 或 Range 分区表避免热点；

7.14 数据删除规范

- 删除表中全部的数据时，使用 TRUNCATE 或者 DROP 后重建方式，不要使用 DELETE。以上几种数据删除方法执行后，都不会立即释放空间，需要等待 TiDB 后台的 GC (garbage collection) 和 Compaction 机制对空间回收后重新利用；
- 对于按范围进行部分数据的删除，如果超过大事务的限制，可以参考以下窗口函数的方法，分成批量小任务进行数据的删除；

a. 将数据按照主键排序，然后调用窗口函数 row_number() 为每一行数据生成行号，接着调用聚合函数按照设置好的页大小对行号进行分组，最终计算出每个分组的行号的最小值和最大值。MySQL [demo]> select min(t.serialno) as start_key, max(t.serialno) as end_key, count() *as page_size from (select* , row_number () over (order by serialno) as row_num from tmp_loan) t group by floor((t.row_num - 1) / 50000) order by start_key; +————+————+————+ | start_key | end_key | page_size | +————+————+————+ | 200000000 | 200050001 | 50000 | | 200050002 | 200100007 | 50000 | | ……. |……. | ……. | | 201900019 | 201950018 | 50000 | | 201950019 | 201999003 | 48985 | +————+————+————+ 40 rows in set (1.51 sec)

b. 借助计算好的分组信息，使用 serialno between start_key and end_key 操作每个分组的数据，实现高效数据删除或者更新。

7.15 分页查询 order by 语法使用规范

- 分页查询语句需要带有排序条件，除业务排序条件外还应包含主键或者其他唯一键以保证分页稳定，规避没有业务排序字段或者一个业务排序字段值匹配多条记录导致结果集不稳定；常规分页语句写法 (start: 起始记录数,page_offset: 每页记录数)：select * from table_a t order by gmt_modified desc,pk limit start，page_offset;

7.16 group by 语法使用规范

- select 字段中不得引用未在 group by 子句中声明的非聚集字段，即不得使用 MySQL non-full group by 语法; 以下的语句是不被允许的，select class,stuname,max(score) as max_score from score group by class;

7.17 模糊匹配 like 语法使用规范

- 使用 like 模糊匹配时，查找字符串中通配符% 放首位会导致无法使用索引。业务语句中使用 like 查找字符串不使用% 放首位，或者使用时结合其他有效的约束条件;

7.18 视图使用规范

- 支持为应用程序建立专门的视图而不必非要应用程序直接访问数据表；
- 视图不可更新，不支持 UPDATE、INSERT、DELETE 等写入操作；

7.19 多表关联查询规范

- 多表关联应该显式使用 join 子句，避免漏掉关联条件，造成笛卡尔积；
- 嵌套 SQL 语句应该为不同表指定不同别名；
- 高并发交易场景，单条语句关联表不超过两张，使用执行计划为 IndexJoin 的多表关联语句，外表建立了正确的条件过滤索引，内表建立了正确的关联和条件过滤索引；
- 低并发的分析场景，单条语句关联表不超过 10 张，其中亿级表不超过 2 张。需要注意 tidb_mem_quota_query 参数指定的单条语句内存使用限制，默认是 1GB，生产建议不超过 16 GB；

7.20 分区表规范

- 当数据需要按时间进行归档清理时，可按某个业务时间段对表进行分区，对分区进行 truncate 操作满足数据清理要求；
- 按时间进行分区表的粒度应该将分区记录数控制在十亿级别，不应该配置太小，如日分区，也不应太大，如年分区；
- 在分区表上进行查找时，查找条件必须包含分区字段的查找条件。分区表的二级索引属于分区内索引，需要先通过分区裁剪的方式，定位到具体分区后再经过二级索引回表查找；

8 TiDB 数据库开发规范

8.1 前言

8.1.1 1. 目的

本文档旨在为使用 TiDB 数据库的应用和系统提供统一规范参考，标准化 TiDB 数据库的开发使用及 SQL 优化流程，提高业务开发系统的规范性和代码的可读性，减轻维护工作量，提高工作效率。

8.1.2 2. 适用范围

- TiDB 数据库开发设计人员
- 数据库管理人员
- 数据库运营人员
- 数据分析人员
- 数据库架构师

8.1.3 3. 注意事项

TiDB 暂时不支持的特性：

- 存储过程
- 触发器
- 自定义函数
- 外键约束
- 全文索引
- 空间索引
- 非 UTF8 字符集

说明

1. 请参考官网文档了解 TiDB 暂不支持的特性列表。

8.2 对象命名规范

用于规范数据库对象的名命，如数据库（DATABASE）、表（TABLE）、索引（INDEX）、用户（USER）等的命名约定。

8.2.1 1. 原则

1. 命名建议使用具有意义的英文词汇，词汇中间以下划线分隔；
2. 命名只能使用英文字母、数字、下划线；
3. 避免用 TiDB 的保留字如：group，order 等作为单个字段名；
4. 建议所有数据库对象使用小写字母。

8.2.2 2. 数据库命名规范

建议按照业务、产品线或者其它指标进行区分，一般不要超过 20 个字符。如：临时库（tmp_crm）、测试库（test_crm）。

8.2.3 3. 表命名规范

1. 同一业务或者模块的表尽可能使用相同的前缀，表名称尽可能表达含义；
2. 多个单词以下划线分隔，不推荐超过 32 个字符；
3. 建议对表的用途进行注释说明，以便于统一认识。如：

- 临时表（tmp_t_crm_relation_0425）
- 备份表（bak_t_crm_relation_20170425）
- 业务运营临时统计表（tmp_st_[业务代码]/创建人缩写/[日期]）
- 账期归档表（t_crm_ec_record_YYYY[MM][DD]）

4. 不同业务模块的表单独建立 DATABASE，并增加相应注释。
5. 目前 TiDB 只支持将 `lower-case-table-names` 值设为 2，即按照大小写来保存表名，按照小写来比较（不区分大小写）。

8.2.4 4 字段命名规范

1. 字段命名需要表示其实际含义的英文单词或简写；
2. 建议各表之间相同意义的字段应同名；
3. 字段也尽量添加注释，枚举型需指明主要值的含义，如”0 - 离线，1 - 在线”；
4. 布尔值列命名为 [is_ 描述]。如 member 表上表示为 enabled 的会员的列命名为 is_enabled；
5. 字段名不建议超过 30 个字符，字段个数不建议大于 60。
6. 尽量避免使用保留字，如 order、from、desc 等，请参考附录部分的官方保留字。

8.2.5 5 索引命名规范

1. 主键索引：pk_[表名称简写]_[字段名简写]
2. 唯一索引：uk_[表名称简写]_[字段名简写]
3. 普通索引：idx_[表名称简写]_[字段名简写]
4. 多单词组成的 column_name，取尽可能代表意义的缩写。

8.3 数据库对象设计

8.3.1 1. 表的设计

1. TiDB 中的一张表的 RowID 是按照主键的字节序排序的（整数类型的主键我们会使用特定的编码使其字节序和按大小排序一致）。使在 CREATE TABLE 语句中不显式地创建主键，TiDB 也会为该表分配一个隐式主键

2. 在进行 TiDB 表设计过程中，有以下几点需要注意

- 为了保障主从集群复制以及增量备份的幂等性，表需要有主键或者唯一索引，需要唯一索引所有字段非空（避免出现多条空值的重复记录）。
- 出于为性能考虑，尽量避免存储超宽表，表字段数建议不超过 60 个，建议单行的总数据大小不要超过 64K，数据长度过大字段最好拆到另外的表。
- 不推荐使用复杂的数据类型。
- 需要 join 的字段，数据类型保障绝对一致，避免隐式类型转换

3. TiDB 字符集默认就是 UTF8 ，而且目前只支持 UTF8

- 如果原文件是 UTF8, 则只需要将编码改为 utf8mb4，此外不需要做其他转换。这是因为 utf8mb4 编码是 utf8 的超集。
- TiDB 中，utf8mb4 的默认排序规则为 utf8mb4_bin（区分大小写）
- TiDB v4.0 及以后的版本，支持 utf8mb4_general_ci（不区分大小写）

8.3.2 2 字段的设计

1. 整数类型

- TiDB 支持 MySQL 所有的整数类型，包括 INTEGER/INT、TINYINT、SMALLINT、MEDIUMINT 以及 BIGINT。
- INT：所有整数类型的字段推荐只使用 INT 或者 BIGINT。
- BIGINT：定义中不推荐添加长度。
- 推荐使用 INT(10) UNSIGNED 存储 IPv4 格式 IP 地址。
- TINYINT(1)、TINYINT(4) 都是存储一个字节，并不会因为括号里的数字改变。例如 TINYINT(4) 存储 22 则会显示 0022，因为最大宽度为 4，达不到的情况下用 0 来补充。

2. 浮点类型

- TiDB 支持 MySQL 所有的浮点类型，包括 FLOAT、DOUBLE、DECIMAL、NUMERIC 等。
- DECIMAL(M,D)：推荐使用 DECIMAL 类型。FLOAT 和 DOUBLE 在存储的时候，存在精度损失的问题，很可能在值的比较时，得到不正确的结果。DECIMAL 在与 VARCHAR、CHAR 类型比较时会转换为 DOUBLE 类型进行比较，也存在精度损失问题，建议在进行比较时使用显式类型转换，如 CAST 避免精度损失。

3. 日期时间类型

- TiDB 支持 MySQL 所有的日期时间类型，包括 DATE、DATETIME、TIMESTAMP、TIME 以及 YEAR。
- DATE：所有只需要精确到天的字段全部使用 DATE 类型，而不应该使用 TIMESTAMP 或者 DATETIME 类型；
- DATETIME：所有需要精确到时间 (时分秒) 的字段均使用 DATETIME，不要使用 TIMESTAMP 类型。TIMESTAMP 支持的精度范围比较小，支持范围为 1970-2038 年，而 DATETIME 支持 1000-9999 年。
- 时间字段使用时间日期类型，不要使用字符串类型存储。否则无法利用日期函数对日期字段进行操作，而需要使用字符串函数进行复杂的操作。

4. 处理日期和时间类型时，请记住下面这些：

- 日期部分必须是按年 -月 -日的顺序 (比如，'1998-09-04')，而不是月 -日 -年或者日 -月 -年的顺序。
  - 如果日期中包含年份为两位数，则这个年份是有歧义的，并不显式地表示实际年份。对于 DATETIME、DATE 和 TIMESTAMP 类型，TiDB 使用如下规则来消除歧义。
    * 将 01 至 69 之间的值转换为 2001 至 2069 之间的值。
    * 将 70 至 99 之间的值转化为 1970 至 1999 之间的值。
- 如果输出格式必须是数值类型，TiDB 会自动将日期或时间值转换为数值类型。例如

```
sql  SELECT NOW(), NOW()+0, NOW(3)+0; +---------------------+---------------+-----
↪  | NOW()    | NOW()+0       | NOW(3)+0          |  +---------------------+--------
↪  | 2012-08-15 09:28:00 | 20120815092800 | 20120815092800.889 |
↪ +---------------------+----------------+--------------------+
```

- 设置不同的 sql_mode 会改变 TiDB 的行为, 以定义 TiDB 支持哪些 SQL 语法及执行哪种数据验证检查。比如 NO_ZERO_DATE 表示 '在严格模式，不要将'0000-00-00' 做为合法日期'。

5. 字符串类型

- TiDB 支持 MySQL 所有的字符串类型，包括 CHAR、VARCHAR、BINARY、VARBINARY、BLOB、TEXT、ENUM 以及 SET。
- VARCHAR(N)：所有动态长度字符串全部使用 VARCHAR 类型，N 表示的是字符数不是字节数，比如 VARCHAR(256)，需要根据实际的宽度来选择 N，N 尽可能小。
- CHAR：仅仅只有单个字符的字段使用 CHAR(1) 类型，例如性别字段。
- TEXT：仅仅当字符数量可能超过 20000 个的时候，才建议使用 TEXT 类型来存放字符类数据，因为所有 TiDB 数据库都会使用 UTF8 字符集。所有使 TEXT 类型的字段建议和原表进行分拆，与原表主键单独组成另外一个表进行存放。
- 不建议使用 ENUM、SET 类型，尽量使用 TINYINT 来代替。
- 当使用宽字段类型（如 Text、MediumBlob、MediumText）时，需注意读取并发度，以控制内存使用预防 OOM 。

8.3.3 3. 字段默认值

1. 在一个数据类型描述中的 DEFAULT value 段描述了一个列的默认值。这个默认值必须是常量，不可以是一个函数或者是表达式。但是对于时间类型，可以例外的使用 NOW、CURRENT_TIMESTAMP、LOCALTIME、LOCALTIMESTAMP 等函数作为 DATETIME 或者 TIMESTAMP 的默认值。
2. BLOB、TEXT 以及 JSON 不可以设置默认值。
3. 如果一个列的定义中没有 DEFAULT 的设置。TiDB 按照如下的规则决定：

- 如果该类型可以使用 NULL 作为值，那么这个列会在定义时添加隐式的默认值设置 DEFAULT NULL。
- 如果该类型无法使用 NULL 作为值，那么这个列在定义时不会添加隐式的默认值设置。

4. 对于一个设置了 NOT NULL 但是没有显式设置 DEFAULT 的列，当 INSERT、REPLACE 没有涉及到该列的值时，TiDB 根据当时的 SQL_MODE 进行不同的行为：

- 如果此时是 strict sql mode，在事务中的语句会导致事务失败并回滚，非事务中的语句会直接报错。
- 如果此时不是 strict sql mode，TiDB 会为这列赋值为列数据类型的隐式默认值。

5. 此时隐式默认值的设置按照如下规则：

- 对于数值类型，它们的默认值是 0。当有 AUTO_INCREMENT 参数时，默认值会按照增量情况赋予正确的值。

- 对于除了时间戳外的日期时间类型，默认值会是该类型的 “零值”。时间戳类型的默认值会是当前的时间。
- TIMESTAMP 和 DATETIME 列可以被自动初始化或者更新为当前时间。
- 对于表里面任意的 TIMESTAMP 或者 DATETIME 列，可以将默认值或者自动更新值指定为 current timestamp，通过在列定义时指定 DEFAULT CURRENT_TIMESTAMP 和 ON UPDATE CURRENT_TIMESTAMP 可以设置这些属性。DEFAULT 也可以指定成某个特定的值，比如 DEFAULT 0 或者 DEFAULT ‘2000-01-01 00:00:00’。

```sql
CREATE TABLE t1 (
ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
dt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP);
```

- 对于除枚举以外的字符串类型，默认值会是空字符串。对于枚举类型，默认值是枚举中的第一个值。

8.3.4 4. 索引的设计

1. 选择区分度大的列建立索引，不在低基数列上建立索引，例如：“性别”，“是否是 XXX”。
2. 单张表的索引数量控制在 5 个以内，避免冗余索引。
3. 索引中的字段数建议不超过 5 个。
4. 唯一索引建议由 3 个或更少的字段组成。
5. 尽量不要在频繁更新的列上创建索引。
6. 尽可能地将使用频率高的，经常被点查使用的列排在复合索引靠前的位置，将经常进行范围查询的列排在后面。
7. 最左前缀匹配原则 (leftmost prefix)，使用联合索引时，从左向右匹配，比如索引 idx_c1_c2_c3 (c1,c2,c3)，相当于创建了 (c1)、(c1,c2)、(c1,c2,c3) 三个索引，where 条件包含上面三种情况的字段比较则可以用到索引，但像 where c1=a and c3=c 只能用到 c1 列的索引，像 c2=b and c3=c 等情况就完全用不到这个索引。
8. 很长的 VARCHAR 字段建立索引时，指定索引长度，没必要对全字段建立索引，根据实际文本区分度决定索引长度即可，例如 idx_table_name (name(10))。
9. 定期删除一些长时间未使用过的索引。
10. ORDER BY，GROUP BY，DISTINCT 的字段需要添加在索引的后面，形成覆盖索引。

11. 新的 select,update,delete 上线，都要先执行 explain 命令，观察执行计划是否有异常情况发现，以确保索引的正确性。

12. 不建议在 where 条件索引列上使用函数，会导致索引失效，如 lower(email)。

13. 使用 like 模糊匹配，% 不要放首位，会导致索引失效。

8.3.5 5. 权限的设计

TiDB 在数据库初始化时会生成一个 'root'@'%' 的默认账户，生产环境建议调整 root 用户为强密码，且不对外开放。线上所需用户建议按照用户或者业务场景划分，根据实际情况对每个用户授予相应权限，例如：

序号	用户名	涵义	用途
1	root	超级用户	全局管理，禁止对外开放
2	dba	数据库管理员	数据库 DBA
3	app	应用开发	应用开发
4	tempuser	临时统计	线上业务临时统计，只读用户
5	other	其他用户	第三方人员访问

8.3.6 6. 约束的设计

在 TiDB 中，开发时需要注意：

1. 主键：

- 一张表只能有一个主键
- 支持定义一个多列组合作为复合主键
- 主键列不能允许 NULL 值
- 对于非索引组织表，可以修改或删除主键。

2. 唯一键：TiDB 中的唯一键可在新建表时添加，也可以在已有表上进行添加和删除操作。

3. 外键：TiDB 不支持外键，仅做语法兼容，不会在 DML 中对外键进行约束检查。外键的级联操作多表数据的功能需要在应用中实现。

注意 TiDB 中 UNIQUE KEY 和 PRIMARY KEY 的区别：

1. Primary key 的一个或多个列必须为 NOT NULL，如果列为 NULL，在增加 PRIMARY KEY 时，列自动更改为 NOT NULL。而 UNIQUE KEY 对列没有此要求。
2. 一个表只能有一个 PRIMARY KEY，但可以有多个 UNIQUE KEY。

8.3.7 7. 分区表的设计

1. 分区类型的选择：需要按照业务场景选择

- TiDB 当前支持的类型包括 Range 分区、List 分区和 Hash 分区等。
- 不同的分区对分区键的数据类型有要求。
- Range 分区和 List 分区可以用于解决业务中大量删除带来的性能问题，支持快速删除分区。- Hash 分区则可以用于大量写入场景下的数据打散。

2. 分区键的选择：

TiDB 要求分区表的每个唯一键（以及主键），必须包含分区表达式中用到的所有列。例如，以下是合法的

```
sql CREATE TABLE t1 ( col1 INT NOT NULL, col2 DATE NOT NULL,
↪ col3 INT NOT NULL, col4 INT NOT NULL, UNIQUE KEY (col1, col2,
↪ col3)) PARTITION BY HASH(col3) PARTITIONS 4;
```

3. SQL 需要适配分区表的写法

TiDB 的分区表支持 SQL 执行计划中的分区裁剪。分区裁剪是指通过分析查询语句中的过滤条件，只选择可能满足条件的分区，不扫描匹配不上的分区，进而显著地减少计算的数据量。为了利用此功能，需要在 SQL 的 WHERE 条件中，带上分区键。Hash 分区表要求等值比较才可能使用 Hash 分区表的裁剪；Range 分区可以支持等值、in 条件、区间比较等实现分区裁剪。

SQL 示例：

```
sql WHERE partition_column = constant 或者 partition_column IN (
↪ constant1, constant2, ..., constantN)
```

8.4 数据模型设计

数据库模型设计是指对于一个给定的应用环境，构造合理的数据库模式，建立数据库及其应用系统，有效存储数据，满足用户信息要求和处理要求。数据库设计在开发过程中处于一个非常重要的地位。一个高效的数据库模型是非常重要和必要的。

8.4.1 1. 完整性

数据库完整性是指数据库中数据的正确性和相容性，数据库完整性是由完整性约束来保证的，数据库完整性对于数据库应用系统非常关键，其作用主要体现在以下几个方面：

- 利用完整性控制机制来实现业务规则，易于定义，容易理解，而且可以降低应用程序的复杂性，提高应用程序的运行效率。

- 合理的数据库完整性设计，能够同时兼顾数据库的完整性和系统的效能。在应用软件的功能测试中，完善的数据库完整性有助于尽早发现应用软件的错误。
- 为了在数据库和应用程序代码之间提供另一层抽象，可以为应用程序建立专门的视图而不必非要应用程序直接访问数据表。这样做还等于在处理数据库变更时给你提供了更多的自由。

8.4.2 2. 性能

性能是衡量一个系统的关键因素，在设计阶段就在性能方面就应该多关注，尽量减少后期的烦恼。在数据库设计阶段，性能上的考虑时需要注意：不能以范式作为唯一标准或者指导，在设计过程中，需要从实际需求出发，以性能提升为根本目标来展开设计工作，一些时候为了提升性能，甚至会做反范式设计。

另外还有一些设计上的方法和技巧：

- 设置合理的字段类型和长度。字段类型在满足需求后应尽量短，比如，能用 INT 就尽量不要用 BIGINT。另外不同数据库在 VARCHAR 和 TEXT 类型在长度和性能上也是不同的，选择时要谨慎。
- 选择高效的主键和索引。由于对表记录的读取都是直接或者间接地通过主键或索引来获取，因此应该该根据具体应用特性来设计合理的主键或索引。同时索引长度的也应该关注，尽量减少索引长度。
- 适度冗余。适度的冗余可以避免关联查询，减少 join 查询。

8.4.3 3. 扩展性

在大规模系统中，除了性能，可扩展性也是设计的关键点，而数据库表扩展性主要包含表逻辑结构、功能字段的增加、分表等。在扩展性上要把握的原则如下：

- 一表一实体。如果不同实体之间有关联时，可增加一个单独的表，不会影响以前的功能。
- 分表设计，也就是水平切分。在 TiDB 目前模型设计阶段应该考虑数据的增长情况，并根据数据特性以及数据之间关系选择合适的切分策略。TiDB 当前支持的类型包括 Range 分区和 Hash 分区等。Range 分区和 List 分区可以用于解决业务中大量删除带来的性能问题；Hash 分区则可以用于大量写入场景下的数据打散。

8.5 SQL 开发规范

8.5.1 1. 建表删表规范

1. 基本原则：表的建立在遵循表命名规范前提下，建议业务应用内部封装建表删表语句增加判断逻辑，防止业务流程异常中断。

2. 详细说明：`create table if not exists table_name` 或者 `drop table if exists table_name` 语句建议增加 if 判断，避免应用侧由于 SQL 命令运行异常造成的异常中断。

8.5.2 2. select * 使用规范

1. 基本原则：避免使用 select * 进行查询。
2. 详细说明：按需求选择合适的字段列，避免盲目地 SELECT * 读取全部字段，因为其会消耗网络带宽。考虑将被查询的字段也加入到索引中，以有效利用覆盖索引功能。

8.5.3 3. 大事务处理

基本原则：避免大事务，TiDB 对单个事务的大小有限制，这层限制是在 KV 层面。反映在 SQL 层面的话，简单来说一行数据会映射为一个 KV entry，每多一个索引，也会增加一个 KV entry。所以这个限制反映在 SQL 层面是：

- 最大单行记录容量为 120MB（TiDB v5.0 及更高的版本可通过 tidb-server 配置项 `performance.txn-entry-size-limit` 调整，低于 v5.0 的版本支持的单行容量为 6MB.
- 支持的最大单个事务容量为 10GB（TiDB v4.0 及更高版本可通过 tidb-server 配置项 `performance.txn-total-size-limit` 调整，低于 TiDB v4.0 的版本支持的最大单个事务容量为 100MB.

另外注意，无论是大小限制还是行数限制，还要考虑事务执行过程中，TiDB 做编码以及事务额外 Key 的开销，在使用的时候，为了使性能达到最优，建议每 100 ~ 500 行写入一个事务。

8.5.4 4. 字段上使用函数规范

1. 基本原则：在取出字段上可以使用相关函数，但是在 Where 条件中的过滤条件字段上避免使用任何函数，包括数据类型转换函数，以避免索引失效。或者可以考虑使用表达式索引功能。
2. 详细说明：

   不推荐的写法：

```
select gmt_create
from ...
where date_format(gmt_create, '%Y%m%d %H:%i:%s') = '20090101 00:00:0'
```

   推荐的写法：

```
select date_format(gmt_create, '%Y%m%d %H:%i:%s')
from .. .
where gmt_create = str_to_date('20090101 00:00:00', '%Y%m%d %H:%i:%s')
```

8.5.5 5. 数据删除规范

1. 基本原则：

   删除表中全部的数据时，推荐使用 TRUNCATE 或者重建（DROP then CREATE 的方式，或者使用分区表中的DROP/TRUNCATE PARTITON 的方式，或者带有条件的DELETE where condition ..limit N的循环方式。

2. 详细说明：

   DELETE，TRUNCATE 和 DROP 都不会立即释放空间。对于 TRUNCATE 和 DROP 操作，在达到 TiDB 的 GC (garbage collection) 时间后（默认 10 分钟. ，TiDB 的 GC 机制会删除数据并释放空间。对于 DELETE 操作 TiDB 的 GC 机制会删除数据，但不会释放空间，而是当后续数据写入 RocksDB 且进行 compact 时对空间重新利用。

8.5.6 6. 其他规范

1. WHERE 条件中不要在索引列上进行数学运算或函数运算；
2. 用 in/union 替换 or，并注意 in 的个数小于 300；
3. 避免使用% 前缀进行模糊前缀查询；
4. 如应用使用 Multi Statements 执行 SQL，即将多个 SQL 使用分号连接，一次性地发给客户端执行，TiDB 只会返回第一个 SQL 的执行结果。
5. 当使用表达式时，检查其是否支持计算下推到存储层的功能 (TiKV、TiFlash)，否则应有预期在 TiDB 层需要消耗更多内存、甚至 OOM。计算下推到存储层的功能列表如下

- TiFlash 支持的计算下推清单
- 下推到 TiKV 的表达式列表
- 谓词下推

8.6 事务限制

8.6.1 1. 隔离级别

TiDB 支持的隔离级别是 RC（Read Committed）与 SI（Snapshot Isolation），其中 SI 与 RR（Repeatable Read）隔离级别基本等价。

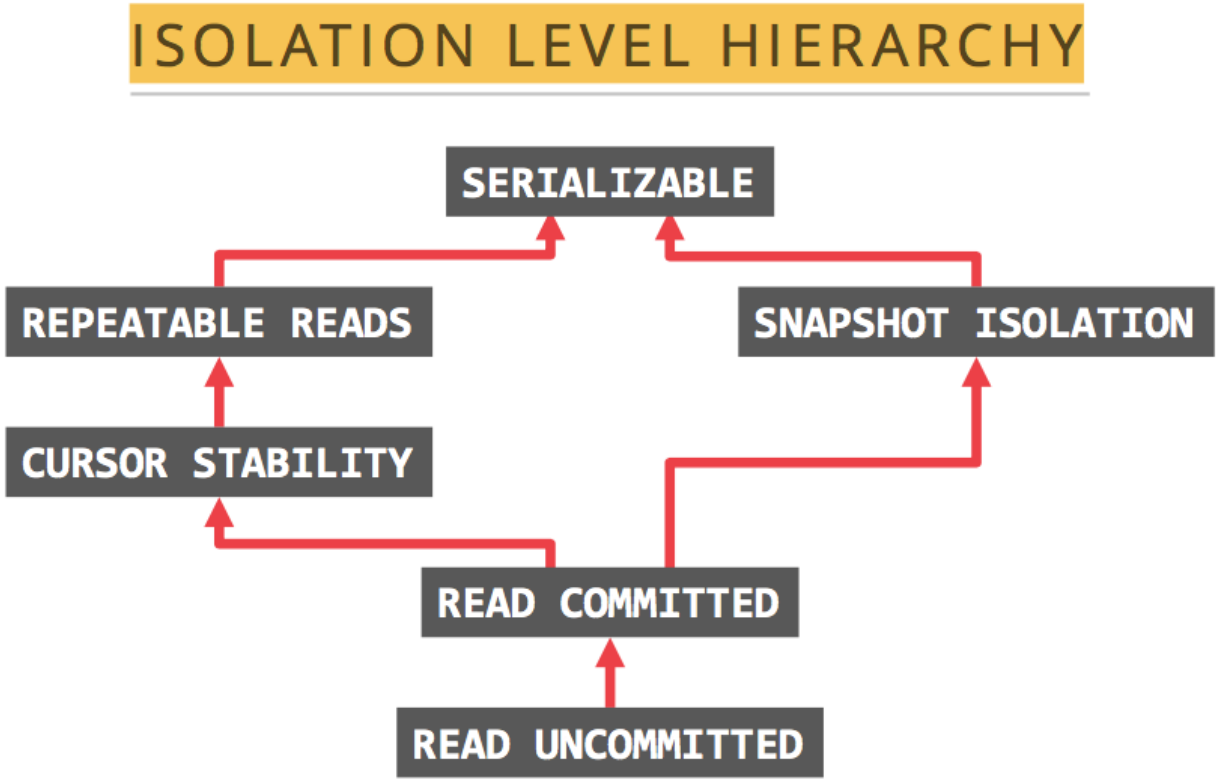


Figure 1: 隔离级别

8.6.2 2. SI 可以克服幻读

TiDB 的 SI 隔离级别可以克服幻读异常（Phantom Reads），但 ANSI/ISO SQL 标准中的 RR 不能。

所谓幻读是指：事务 A 首先根据条件查询得到 n 条记录，然后事务 B 改变了这 n 条记录之外的 m 条记录或者增添了 m 条符合事务 A 查询条件的记录，导致事务 A 再次发起请求时发现有 n+m 条符合条件记录，就产生了幻读。

例如：系统管理员 A 将数据库中所有学生的成绩从具体分数改为 ABCDE 等级，但是系统管理员 B 就在这个时候插入了一条具体分数的记录，当系统管理员 A 改结束后发现还有一条记录没有改过来，就好像发生了幻觉一样，这就叫幻读。

8.6.3 3. SI 不能克服写偏斜

TiDB 的 SI 隔离级别不能克服写偏斜异常（Write Skew），需要使用 Select for update 语法来克服写偏斜异常。

写偏斜异常是指两个并发的事务读取了不同但相关的记录，接着这两个事务各自更新了自己读到的数据，并最终都提交了事务，如果这些相关的记录之间存在着不能被多个事务并发修改的约束，那么最终结果将是违反约束的。

例如：值班表有两列，姓名以及值班状态，0 代表不值班，1 代表值班

姓名	值班状态
张三	0
李四	0
王五	0

有这样一个事务，它的逻辑是判断当前无人值班，则分配一个值班人。当该程序顺序执行时，只会分配一个值班人。但当它并行执行时，就可能出现多人同时为值班状态的错误，造成这个错误的原因就是写偏斜。

8.6.4 4. 不支持 savepoint 和嵌套事务

Spring 支持的 PROPAGATION_NESTED 传播行为会启动一个嵌套的事务，它是当前事务之上独立启动的一个子事务。嵌套事务开始时会记录一个 savepoint ，如果嵌套事务执行失败，事务将会回滚到 savepoint 的状态。嵌套事务是外层事务的一部分，它将会在外层事务提交时一起被提交。下面案例展示了 savepoint 机制：

```
mysql> BEGIN;
mysql> INSERT INTO T2 VALUES(100);
mysql> SAVEPOINT svp1;
mysql> INSERT INTO T2 VALUES(200);
mysql> ROLLBACK TO SAVEPOINT svp1;
mysql> RELEASE SAVEPOINT svp1;
mysql> COMMIT;
mysql> SELECT * FROM T2;
+------+
| ID   |
+------+
| 100 |
+------+
```

TiDB 不支持 savepoint 机制，因此也不支持 PROPAGATION_NESTED 传播行为。基于 Java Spring 框架的应用如果使用了 PROPAGATION_NESTED 传播行为，需要在应用端做出调整，将嵌套事务的逻辑移除。

8.6.5 5. 大事务限制

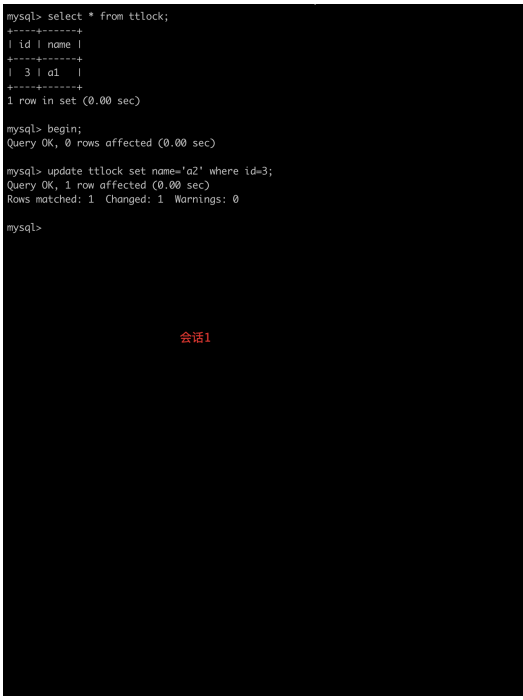
基本原则是要限制事务的大小。TiDB 对单个事务的大小有限制，这层限制是在 KV 层面。反映在 SQL 层面的话，简单来说一行数据会映射为一个 KV entry，每多一个索引，也会增加一个 KV entry。所以这个限制反映在 SQL 层面是：

- 最大单行记录容量为 120MB (TiDB v5.0 及更高的版本可通过 tidb-server 配置项 `performance.txn-entry-size-limit` 调整，低于 TiDB v5.0 的版本支持的单行容量为 6MB)

- 支持的最大单个事务容量为 10GB（TiDB v4.0 及更高版本可通过 tidb-server 配置项 `performance.txn-total-size-limit` 调整，低于 TiDB v4.0 的版本支持的最大单个事务容量为 100MB）

另外注意，无论是大小限制还是行数限制，还要考虑事务执行过程中，TiDB 做编码以及事务额外 Key 的开销。在使用的时候，为了使性能达到最优，建议每 100 ~ 500 行写入一个事务。

8.6.6 6. 自动提交的 SELECT FOR UPDATE 语句不会等锁



自动提交下的 select for update 目前不会加锁。效果如下图所示：这是已知的与 MySQL 不兼容的地方。可以通过使用显式的 `begin;commit;` 解决该问题。

8.7 隐式类型转换

8.7.1 1. 转换规则

当 SQL 中谓词两侧的数据类型不一致时，TiDB 将隐式地将一侧或两侧的数据类型进行转换，将其变为兼容的数据类型，以进行谓词运算。

TiDB 中隐式类型转换规则如下：

1. 如果一个或两个参数都是 NULL，比较的结果是 NULL（NULL 安全的 <=> 相等比较运算符除外，对于 NULL <=> NULL，结果为 true，不需要转换）。
2. 如果比较操作中的两个参数都是字符串，则将它们作为字符串进行比较。
3. 如果两个参数都是整数，则将它们作为整数进行比较。

4. 如果不与数字进行比较，则将十六进制值视为二进制字符串。
5. 如果其中一个参数是十进制值，则比较取决于另一个参数。如果另一个参数是十进制或整数值，则将参数与十进制值进行比较，如果另一个参数是浮点值，则将参数与浮点值进行比较
6. 如果其中一个参数是 TIMESTAMP 或 DATETIME 列，另一个参数是常量，则在执行比较之前将常量转换为时间戳。
7. 在所有其他情况下，参数都是作为浮点数（DOUBLE 类型）比较的。

8.7.2 2. 隐式类型转换引起的后果

隐式类型转换增强了人机交互的易用性，但在应用代码中，我们应尽量避免隐式类型转换出现，这是由于隐式类型转换会导致：

- 索引失效
- 精度丢失

8.7.2.1 1. 索引失效

如下案例，account_id 为主键，其数据类型为 varchar。通过执行计划可见，该 SQL 发生了隐式类型转换，无法使用索引。

```
3 rows in set (0.00 sec)desc select * from account where account_id
    ↪ =6010000000009801;
+--
    ↪ ----------------------+---------------+----------+--------------+----------
    ↪
| id                    | estRows       | task     | access object | operator
    ↪ info                                  |
+--
    ↪ ----------------------+---------------+----------+--------------+----------
    ↪
| TableReader_7         | 8000628000.00 | root     |               | data:
    ↪ Selection_6                           |
|  ─Selection_6         | 8000628000.00 | cop[tikv] |              | eq(cast(
    ↪ findpt.account.account_id), 6.010000000009801e+15) |
|    ─TableFullScan_5   | 10000785000.00 | cop[tikv] | table:account | keep
    ↪ order:false                           |
+--
    ↪ ----------------------+---------------+----------+--------------+----------
    ↪
3 rows in set (0.00 sec)
```

说明：从以上执行计划中，可见 Cast 算子

8.7.2.2 2. 精度丢失

如下案例，字段 a 的数据类型为 decimal(32,0)，从执行计划可以得知，出现了隐式类型转换，decimal 字段和字符串常值都被转换为 double 类型，而 double 类型的精度没有 decimal 高，出现了精度丢失，在这个 case 中，造成了筛选出范围之外的结果集的错误。

```
desc select * from t1 where a between '12123123' and '
    ↪ 111122221111111200000';
+--
    ↪ ----------------------+---------+----------+--------------+------------------
    ↪
| id                    | estRows | task    | access object | operator info
    ↪                                                          |
+--
    ↪ ----------------------+---------+----------+--------------+------------------
    ↪
| TableReader_7         | 0.80    | root     |               | data:Selection_6
    ↪                                                          |
|  ─Selection_6         | 0.80    | cop[tikv] |              | ge(cast(findpt.t1
    ↪ .a), 1.2123123e+07), le(cast(findpt.t1.a), 1.1112222111111112e+21) |
|    ─TableFullScan_5   | 1.00    | cop[tikv] | table:t1     | keep order:false,
    ↪  stats:pseudo                                          |
+--
    ↪ ----------------------+---------+----------+--------------+------------------
    ↪
3 rows in set (0.00 sec)
```

说明：从以上执行计划中，可见 Cast 算子

```
MySQL [findpt]> select * from t1 where a between '12123123' and '
    ↪ 111122221111111200000';
+-----------------------+
| a                     |
+-----------------------+
| 111122221111111222211 |
+-----------------------+
1 row in set (0.01 sec)
```

说明：以上执行出现了错误结果

8.8 结果集不稳定

8.8.1 1. group by

出于便捷的考量，MySQL “扩展” 了 group by 语法，使 select 子句可以引用未在 group by 子句中声明的非聚集字段，也就是 non-full group by 语法。在其他数据库中，这被认为

是一种语法错误，因为这会导致结果集不稳定。

在下例的 3 条 SQL 语句中，第一条 SQL 使用了 full group by 语法，所有在 select 子句中引用的字段，都在 group by 子句中有所声明，所以它的结果集是稳定的，可以看到 class 与 stuname 的全部组合共有三种；第二条与第三条是同一个 SQL，但它在两次执行时得到了不同的结果，这条 SQL 的 group by 子句中仅声明了一个 class 字段，因此结果集只会针对 class 进行聚集，class 的唯一值有两个，也就是说结果集中只会包含两行数据，而 class 与 stuname 的全部组合共有三种，班级 2018_CS_03 有两位同学，每次执行时返回哪位同学是没有语义上的限制的，都是符合语义的结果。

```
mysql> select a.class, a.stuname, max(b.courscore) from stu_info a join
    ↪ stu_score b on a.stuno=b.stuno group by a.class, a.stuname order by a
    ↪ .class, a.stuname;
+-----------+-------------+-----------------+
| class     | stuname     | max(b.courscore) |
+-----------+-------------+-----------------+
| 2018_CS_01 | MonkeyDLuffy |        95.5 |
| 2018_CS_03 | PatrickStar |        99.0 |
| 2018_CS_03 | SpongeBob |          95.0 |
+-----------+-------------+-----------------+
3 rows in set (0.00 sec)

mysql> select a.class, a.stuname, max(b.courscore) from stu_info a join
    ↪ stu_score b on a.stuno=b.stuno group by a.class order by a.class, a.
    ↪ stuname;
+-----------+-------------+-----------------+
| class     | stuname     | max(b.courscore) |
+-----------+-------------+-----------------+
| 2018_CS_01 | MonkeyDLuffy |        95.5 |
| 2018_CS_03 | SpongeBob |          99.0 |
+-----------+-------------+-----------------+
2 rows in set (0.01 sec)

mysql> select a.class, a.stuname, max(b.courscore) from stu_info a join
    ↪ stu_score b on a.stuno=b.stuno group by a.class order by a.class, a.
    ↪ stuname;
+-----------+-------------+-----------------+
| class     | stuname     | max(b.courscore) |
+-----------+-------------+-----------------+
| 2018_CS_01 | MonkeyDLuffy |        95.5 |
| 2018_CS_03 | PatrickStar |        99.0 |
+-----------+-------------+-----------------+
2 rows in set (0.01 sec)
```

因此，想保障 group by 语句结果集的稳定，请使用 full group by 语法。

MySQL 提供了一个 SQL_MODE 开关 ONLY_FULL_GROUP_BY 来控制是否进行

full group by 语法的检查，TiDB 也兼容了这个 SQL_MODE 开关：

```
mysql> select a.class, a.stuname, max(b.courscore) from stu_info a join
    ↪ stu_score b on a.stuno=b.stuno group by a.class order by a.class, a.
    ↪ stuname;
+-----------+-------------+-----------------+
| class     | stuname     | max(b.courscore) |
+-----------+-------------+-----------------+
| 2018_CS_01 | MonkeyDLuffy |        95.5 |
| 2018_CS_03 | PatrickStar |        99.0 |
+-----------+-------------+-----------------+
2 rows in set (0.01 sec)

mysql> set @@sql_mode='STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION,
    ↪ ONLY_FULL_GROUP_BY';
Query OK, 0 rows affected (0.01 sec)

mysql> select a.class, a.stuname, max(b.courscore) from stu_info a join
    ↪ stu_score b on a.stuno=b.stuno group by a.class order by a.class, a.
    ↪ stuname;
ERROR 1055 (42000): Expression #2 of ORDER BY is not in GROUP BY clause and
    ↪  contains nonaggregated column '' which is not functionally dependent
    ↪  on columns in GROUP BY clause; this is incompatible with sql_mode=
    ↪ only_full_group_by
```

说明：上例为 sql_mode 设置了 ONLY_FULL_GROUP_BY 的效果

8.8.2 2. order by

在 SQL 的语义中，只有使用了 order by 语法才会保障结果集的顺序输出。而单机数据库由于数据都存储在一台服务器上，在不进行数据重组时，多次执行的结果往往是稳定的，有些数据库 (尤其是 MySQL InnoDB 存储引擎) 还会按照主键或索引的顺序进行结果集的输出。TiDB 是分布式数据库，数据被存储在多台服务器上，另外 TiDB 层不缓存数据页，因此不含 order by 的 SQL 语句的结果集展现顺序容易被感知到不稳定。想要按顺序输出的结果集，需明确地把要排序的字段添加到 order by 子句中，这符合 SQL 的语义。

在下面的案例中，用户只在 order by 子句中添加了一个字段，TiDB 只会按照这一个字段进行排序。

```
mysql> select a.class, a.stuname, b.course, b.courscore from stu_info a
    ↪ join stu_score b on a.stuno=b.stuno order by a.class;
+-----------+-------------+------------------------+-----------+
| class     | stuname     | course                 | courscore |
+-----------+-------------+------------------------+-----------+
| 2018_CS_01 | MonkeyDLuffy | PrinciplesofDatabase | 60.5 |
| 2018_CS_01 | MonkeyDLuffy | English              |     43.0 |
```

```
| 2018_CS_01 | MonkeyDLuffy | OpSwimming          |     67.0 |
| 2018_CS_01 | MonkeyDLuffy | OpFencing           |     76.0 |
| 2018_CS_01 | MonkeyDLuffy | FundamentalsofCompiling | 88.0 |
| 2018_CS_01 | MonkeyDLuffy | OperatingSystem     |     90.5 |
| 2018_CS_01 | MonkeyDLuffy | PrincipleofStatistics | 69.0 |
| 2018_CS_01 | MonkeyDLuffy | ProbabilityTheory   |     76.0 |
| 2018_CS_01 | MonkeyDLuffy | Physics             |     63.5 |
| 2018_CS_01 | MonkeyDLuffy | AdvancedMathematics |   95.5 |
| 2018_CS_01 | MonkeyDLuffy | LinearAlgebra       |     92.5 |
| 2018_CS_01 | MonkeyDLuffy | DiscreteMathematics |   89.0 |
| 2018_CS_03 | SpongeBob | PrinciplesofDatabase |     88.0 |
| 2018_CS_03 | SpongeBob | English             |     79.0 |
| 2018_CS_03 | SpongeBob | OpBasketball        |     92.0 |
| 2018_CS_03 | SpongeBob | OpTennis            |     94.0 |
| 2018_CS_03 | PatrickStar | LinearAlgebra     |      6.5 |
| 2018_CS_03 | PatrickStar | AdvancedMathematics |    5.0 |
| 2018_CS_03 | SpongeBob | DiscreteMathematics |     72.0 |
| 2018_CS_03 | PatrickStar | ProbabilityTheory |     12.0 |
| 2018_CS_03 | PatrickStar | PrincipleofStatistics | 20.0 |
| 2018_CS_03 | PatrickStar | OperatingSystem   |     36.0 |
| 2018_CS_03 | PatrickStar | FundamentalsofCompiling | 2.0 |
| 2018_CS_03 | PatrickStar | DiscreteMathematics |   14.0 |
| 2018_CS_03 | PatrickStar | PrinciplesofDatabase |    9.0 |
| 2018_CS_03 | PatrickStar | English           |     60.0 |
| 2018_CS_03 | PatrickStar | OpTableTennis     |     12.0 |
| 2018_CS_03 | PatrickStar | OpPiano           |     99.0 |
| 2018_CS_03 | SpongeBob | FundamentalsofCompiling | 43.0 |
| 2018_CS_03 | SpongeBob | OperatingSystem     |     95.0 |
| 2018_CS_03 | SpongeBob | PrincipleofStatistics |   90.0 |
| 2018_CS_03 | SpongeBob | ProbabilityTheory   |     87.0 |
| 2018_CS_03 | SpongeBob | Physics             |     65.0 |
| 2018_CS_03 | SpongeBob | AdvancedMathematics |     55.0 |
| 2018_CS_03 | SpongeBob | LinearAlgebra       |     60.5 |
| 2018_CS_03 | PatrickStar | Physics           |      6.0 |
+------------+--------------+-------------------------+-----------+
36 rows in set (0.01 sec)
```

当遇到相同的 order by 值时，排序结果不稳定。为减少随机性，应当尽可能保持 order by 值的唯一性。不能保证唯一的继续加，保证 order by 的字段组合是唯一时，结果才能唯一。

8.8.3 3. 由于 group_concat() 中没有使用 order by 导致结果集不稳定

结果集不稳定是因为 TiDB 是并行地从存储层读取数据，所以 group_concat() 在不加 order by 的情况下得到的结果集展现顺序容易被感知到不稳定。

group_concat() 要获取到按顺序输出的结果集，需要把用于排序的字段添加到 order by 子句中，这样才符合 SQL 的语义。在下面的案例中，使用 group_concat() 不加 order by 的情况下拼接 customer_id ，造成结果集不稳定：

1. 不加 order by

   第一次查询：

```
mysql> select GROUP_CONCAT( customer_id SEPARATOR ',' ) FROM customer
    ↪ where customer_id like '200002%';
+--
    ↪ ------------------------------------------------------------------------+
    ↪
| GROUP_CONCAT(customer_id SEPARATOR ',')                                      |
+--
    ↪ ------------------------------------------------------------------------+
    ↪
|
    ↪ 20000200992,20000200993,20000200994,20000200995,20000200996,20000200...
    ↪  |
+--
    ↪ ------------------------------------------------------------------------+
    ↪
```

   第二次查询：

```
mysql> select GROUP_CONCAT( customer_id SEPARATOR ',' ) FROM customer
    ↪ where customer_id like '200002%';
+--
    ↪ ------------------------------------------------------------------------+
    ↪
| GROUP_CONCAT(customer_id SEPARATOR ',')                                      |
+--
    ↪ ------------------------------------------------------------------------+
    ↪
|
    ↪ 20000203040,20000203041,20000203042,20000203043,20000203044,20000203...
    ↪  |
+--
    ↪ ------------------------------------------------------------------------+
    ↪
```

2. 加 order by

   第一次查询：

```
mysql> select GROUP_CONCAT( customer_id order by customer_id SEPARATOR
    ↪ ',' ) FROM customer where customer_id like '200002%';
```

```
+--
    ↪ ----------------------------------------------------------------------+
    ↪
| GROUP_CONCAT(customer_id SEPARATOR ',')                                    |
+--
    ↪ ----------------------------------------------------------------------+
    ↪
|
    ↪ 20000200000,20000200001,20000200002,20000200003,20000200004,20000200...
    ↪  |
+--
    ↪ ----------------------------------------------------------------------+
    ↪
```

第二次查询：

```
mysql> select GROUP_CONCAT( customer_id order by customer_id SEPARATOR
    ↪ ',' ) FROM customer where customer_id like '200002%';
+--
    ↪ ----------------------------------------------------------------------+
    ↪
| GROUP_CONCAT(customer_id SEPARATOR ',')                                    |
+--
    ↪ ----------------------------------------------------------------------+
    ↪
|
    ↪ 20000200000,20000200001,20000200002,20000200003,20000200004,20000200...
    ↪  |
+--
    ↪ ----------------------------------------------------------------------+
    ↪
```

8.8.4 4. select * from t limit n 的结果不稳定

返回结果与数据在 TiKV 节点上的分布有关。如果进行了多次查询，TiKV 不同 region 返回结果的速度不同，会造成结果不稳定。

8.9 索引的使用注意

与其他的 RDBMS 一样，TiDB 需要通过索引才能保障数据读取的性能，而 TiDB 往往被用于承载单机 RDBMS 难以承载的数据量，因此索引尤其是复合索引的设计尤为关键。

8.9.1 1. TiDB 中的索引

索引也是数据，也要占用存储空间。和表中的数据一样，TiDB 中表的索引在存储引擎中也被作为 kv 来存储，一行索引是一个 kv 对。例如一张有 10 个索引的表，每插入一行数据的时候，会写入 11 个 kv 对。

TiDB 支持主键索引，唯一索引，也支持二级索引，构成以上索引的可以是单一列，也可以是多个列（复合索引）。

TiDB 目前（v5.0）还不支持反向/双向索引，全文索引，分区表的全局索引。

TiDB 中在查询的谓词是 =，>，<，>=，<=，like '...%'，not like '...%'，in，not in，<>，!=，is null，<=>，is not null，between...and ... 时能够使用索引，使用与否由优化器来决策。

TiDB 中在查询的谓词是 like '%...'，like '%...%'，not like '%...'，not like '%...%' 时，都无法使用索引。

8.9.2 2. 复合索引的设计

TiDB 中的复合索引形如 `key indexkeyname (a,b,c)`，与其他数据库一样，设计复合索引的一般原则是尽可能的把使用频率比较高的字段放在前面。使用中需要特别注意，复合索引中前一列的范围查询会中止后续索引列的使用，可以通过下面的案例来理解这个特性。在如下的查询中：

```
select a,b,c from tablename where a<predicate>' <value1>'  and b<predicate
    ↪ >' <value2>'  and c<predicate>' <value3>' ;
```

如果 a 条件的谓词（语句中的 predicate）是 = 或 in，那么在 b 的查询条件上就可以利用到组合索引 (a,b,c)。例：`select a,b,c from tablename where a**=**1 and b\<5 ↪ and c=' abc' ;`

同样的，如果 a 条件和 b 条件的谓词都是 = 或 in，那么在 c 上的查询就可以利用到组合索引 (a,b,c)。例：`select a,b,c from tablename where a **in** (1,2,3)and ↪ b**=**5 and c=' abc' ;`

如果 a 条件的谓词不是 = 也不是 in，那么 b 上的查询就无法利用到组合索引 (a,b,c)。此时 b 条件将在 a 条件筛选后的数据中进行无索引的数据扫描。例：`select a,b,c from ↪ tablename where a**\>**1 and b\<5 and c=' abc' ;`

这是由于在 TiDB 中，复合索引中排在前面的列如果被用于范围查询，那么后续列的查询就会在前一列筛选后的数据范围中进行非索引的扫描。

综上，在 TiDB 中进行复合索引设计时，需要尽可能的将使用频率高的，经常被点查使用的列排在前面，将经常进行范围查询的列排在后面。

另外形如 `select c, count(*)from tabname where a=1 and b=2 group by c ↪ order by c;` 的查询可以利用到索引 (a,b,c)，同样遵循上面的原则。

8.10 自增列的使用注意

8.10.1 1. 原理

TiDB 的自增 ID (auto_increment) 只保证自增且唯一，并不保证连续分配。TiDB 目前采用批量分配的方式，所以如果在多台 TiDB 上同时插入数据，分配的自增 ID 会不连续。当多个线程并发往不同的 tidb-server 插入数据的时候，有可能会出现后插入的数据自增 ID 小的情况。此外，TiDB 允许给数值类型的列指定 auto_increment，且一个表只允许一个属性为 auto_increment 的列。

8.10.2 2. 最佳实践

设置自增 ID 的目的一般是将它作为表内数据的唯一性约束，因此被设计为主键或唯一索引，此类列属性应带有 `not null`。

自增列可以定义在整数、FLOAT 或 DOUBLE 的列上；但是最佳实践是使用整型。

在几种整型类型中，我们建议使用 BIGINT，这是由于即使在单机数据库中也屡见 INT 类型的自增 ID 被耗光的情况，而 TiDB 被用于处理比单机数据大得多的数据量。此外 TiDB 采用多线程的方式分配自增 ID，因此 INT 类型无法满足需求。另外自增 ID 一般不需要存储负值，为列增加 unsigned 属性可以扩充一倍的 id 存储容量。INT 无符号的范围是 0 到 4294967295，BIGINT 无符号的范围是 0 到 18446744073709551615。

综上，自增 ID 设计的最佳实践如下：

```
`auto_inc_id` bigint unsigned not null unique key auto_increment comment '
    ↪ 自增 ID'
```

8.10.3 3. 显式赋值的后果及挽救措施

TiDB 实现自增 ID 的原理是每个 tidb-server 实例缓存一段 ID 值用于分配（目前会缓存 30000 个 ID），用完这段值再去取下一段。

在集群中有多个 tidb-server 时，人为向自增列写入值之后，可能会导致 TiDB 分配自增值冲突而报 “Duplicate entry” 错误：

假设有这样一个带有自增 ID 的表：`create table t(id int unique key
↪ auto_increment,c int);`

假设集群中有两个 tidb-server 实例 A 和 B（A 缓存 [1,30000] 的自增 ID，B 缓存 [30001,60000] 的自增 ID），依次执行如下操作：

客户端向 B 插入一条将 id 设置为 1 的语句 `insert into t values (1,1)`，并执行成功。

客户端向 A 发送 Insert 语句 `insert into t (c)(1)`，这条语句中没有指定 id 的值，所以会由 A 分配，当前 A 缓存了 [1, 30000] 这段 ID，所以会分配 1 为自增 ID 的值，并把本地计数器加 1。而此时数据库中已经存在 id 为 1 的数据，最终返回 `Duplicated Entry` 错误。

处理该问题只需要调大表上的 AUTO_INCREMENT 属性值即可让所有 tidb-server 重新获取一段自增 ID：

1. 确认表上自增值的最大值：`show create table t;`
2. 修改表上的自增值最大值到一个更大的值：`alter table t AUTO_INCREMENT =120000;`

自增列的另外一个问题是：因为每个显式赋值，都需要 reset auto_increment 动作，这会体现在 SQL 的 prepare 时间变长。这一点尤其在批量数据操作中体现明显，需要避免这样的使用方式。

8.11 TiDB 中的各种超时

8.11.1 1. GC 超时

TiDB 的事务的实现采用了 MVCC（多版本并发控制）机制，当新写入的数据覆盖旧的数据时，旧的数据不会被替换掉，而是与新写入的数据同时保留，并以时间戳来区分版本。TiDB 通过定期 GC 的机制来清理不再需要的旧数据。

默认配置下 TiDB 可以保障每个 MVCC 版本（一致性快照）保存 10 分钟，读取时间超过 10 分钟的事务，会收到报错 `GC life time is shorter than transaction duration`

当用户确信自己需要更长的读取时间时，比如在使用了 Mydumper 做全量备份的场景中（Mydumper 备份的是一致性的快照），可以通过调整 TiDB 中 `mysql.tidb` 表中的 `tikv_gc_life_time` 的值来调大 MVCC 版本保留时间，需要注意的是 tikv_gc_life_time 的配置是立刻影响全局的，调大它会为当前所有存在的快照增加生命时长，调小它会立即缩短所有快照的生命时长。过多的 MVCC 版本会拖慢 TiKV 的处理效率，在使用 Mydumper 做完全量备份后需要及时把 tikv_gc_life_time 调整回之前的设置。

更多关于 GC 的信息，请参考官网文档：https://pingcap.com/docs-cn/stable/reference/garbage-collection/overview/

8.11.2 2. 事务超时

垃圾回收 (GC) 不会影响到正在执行的事务。但悲观事务的运行仍有上限，有基于事务超时的限制（TiDB 配置文件 [performance] 类别下的 `max-txn-ttl` 修改，默认为 60 分钟）和基于事务使用内存的限制。

形如 `insert into t10 select * from t1` 的 SQL 语句，不会受到 GC 的影响，但超过了 `max-txn-ttl` 的时间后，会由于超时而回滚。

8.11.3 3. SQL 执行时间超时

TiDB 还提供了一个系统变量来限制单条 SQL 语句的执行时间：max_execution_time，它的默认值为 0，表示无限制。`max_execution_time` 目前对所有类型的 statement 生效，并非只对 SELECT 语句生效。其单位为 ms，但实际精度在 100ms 级别，而非更准确的毫秒级别。

8.11.4 4. JDBC 查询超时

MySQL jdbc 的查询超时设置 `setQueryTimeout()` 对 TiDB 不起作用。这是因为现实客户端感知超时时，向数据库发送一个 KILL 命令。但是由于 tidb-server 是负载均衡的，为防止在错误的 tidb-server 上终止连接，tidb-server 不会执行这个 KILL。这时就要用 `MAX_EXECUTION_TIME` 实现查询超时的效果。

TiDB 提供了三个与 MySQL 兼容的超时控制参数：

1. `wait_timeout`, 控制与 Java 应用连接的非交互式空闲超时时间, 默认值为 0，即允许连接无限闲置。
2. `interactive_timeout`，控制与 Java 应用连接的交互式空闲超时时间，默认值为 8 小时。
3. `max_execution_time`，控制连接中 SQL 执行的超时时间，默认值是 0，即允许连接无限忙碌（一个 SQL 语句执行无限的长的时间）。

但在实际生产环境中，空闲连接和一直无限执行的 SQL 对数据库和应用都有不好的影响。你可以通过在应用的连接字符串中配置这两个 session 级的变量来避免空闲连接和执行时间过长的 SQL 语句。例如，设置 `sessionVariables=wait_timeout=3600`（1 小时）和 `sessionVariables=max_execution_time=300000`（5 分钟）。

8.12 JDBC 最佳实践

8.12.1 1. MySQL Connector/J 推荐版本

TiDB 服务端兼容 MySQL 5.7，客户端推荐使用 5.1.36 或更高版本的 5.1.x jdbc 驱动 下载链接。

8.12.2 2. JDBC 参数设置

Java 应用常用的数据库连接池包括 weblogic、c3p0、Druid 等。使用连接池配置数据源时，需要配置一系列参数，其中比较重要的包括 jdbc 的 url 配置，超时探活机制等。充分认识并理解各项参数有助于让 TiDB 发挥出更高的性能。MySQL 5.1 版本 jdbc configuration properties 参见 链接：

一个建议的 url 配置如下：

```
spring.datasource.url=JDBC:mysql://{TiDBIP}:{TiDBPort}/{DBName}?
    ↪ characterEncoding=utf8&useSSL=false&useServerPrepStmts=true&
    ↪ prepStmtCacheSqlLimit=10000000000&useConfigs=maxPerformance&
    ↪ rewriteBatchedStatements=true&defaultfetchsize=-214783648
```

8.13 缓解热点问题

8.13.1 1. 写入热点

大部分单机 RDBMS 采用 B+ tree 数据结构，主键往往是用于组织数据的关键索引 (此时表被称作索引组织表)，同一数据页内的记录按主键顺序存放。因此单机 RDBMS 产品一般推荐写入连续的序列号，这样每次写入新的记录，都会顺序添加到当前 B+ tree 索引节点的后续位置，当前的数据页写满时，会自动开始新一页的写入。相反，过于随机的主键值，会导致新记录被写入到数据页的某个中间位置，造成数据的移动而带来了额外的开销。

虽然 TiDB 具有不同于单机 RDBMS 的数据结构，但顺序的主键值写入，在 TiDB 上也会产生类似的效果：TiKV 上一个的 Region 被写满，进而分裂出一个新的 Region ，后续的写入转由新的 Reigon 来承载。但甲之蜜糖，乙之砒霜，单机 RDBMS 的最佳实践放到 TiDB 上，会使写入压力总是集中在一个 Region 上，这样就构成了持续的写入热点，无法发挥出 TiDB 这种分布式数据库的并行写入能力，降低了业务写入瓶颈，造成了系统资源的浪费。

TiDB 发生写入热点的原因主要有以下几种：

- 小表，整个表只有一个 Region ，高并发的请求会造成读写热点。
- 大量写入时 Key 值的离散程度不足以跳过一个 Region 大小 (96 MB) 而引起写入热点。

8.13.2 2. 热点的观测和定位

从 TiDB v4.0 版本开始提供便于迅速识别集群负载的 Dashboard 流量可视化页面 (Key Visualizer)，下图展示了写入热点的显示效果，中间一条明亮的曲线即标志着存在一张连续写入 Key 值的表。而右上侧的一组线条则显示出一个写入压力较为均匀的负载。Key Visualizer 的具体使用方法请参考官方文档。

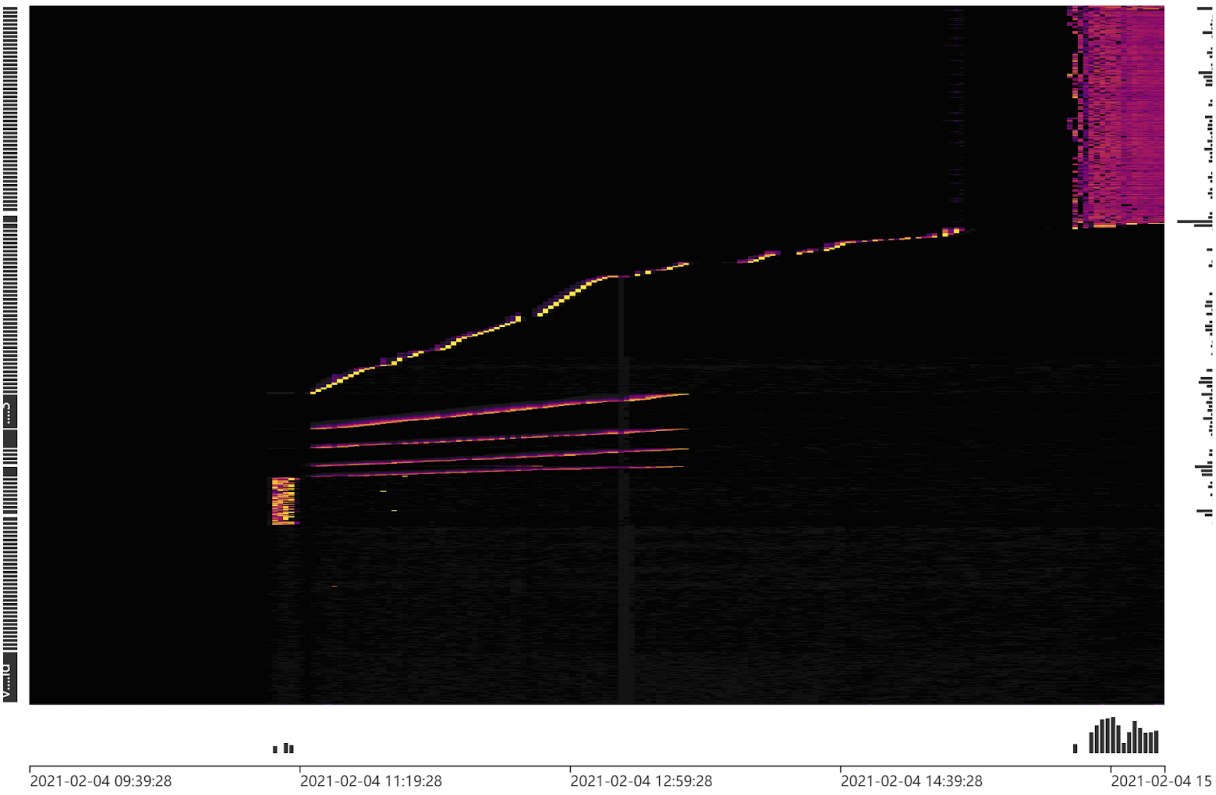


Figure 2: 热力图

图 1. 写入热点在 Dashboard Key Visualizer 中的显示效果

8.13.3 3. 写入热点的缓解

1. 配置 SHARD_ROW_ID_BITS 参数打散写入热点

   对于非索引组织表（默认行为），TiDB 会使用一个隐式的自增 rowid，大量 INSERT 时会把数据集中写入单个 Region，造成写入热点。通过设置 SHARD_ROW_ID_BITS 可以把 rowid 打散写入多个不同的 Region，缓解写入热点问题。但是设置的过大会造成 RPC 请求数放大，增加 CPU 和网络开销。需注意：通过其他方式设计的随机主键，当主键过分打散时，也会存在同样的问题。其他方式生成的过于随机的主键（比如使用 UUID）也会有此问题。这是因为尽管其离散程度很高，但是也会因此带来网络层低效攒批、RocksDB compaction 时的写入放大、Regions 的过量激活问题，给整体 NETWORK 和 IO 和 CPU 都带来压力。

   用 SHARD_ROW_ID_BITS 来设置隐藏列 _tidb_rowid 分片数量的 bit 位数，默认值为 0，即 2^0 = 1 个分片。SHARD_ROW_ID_BITS = 4 代表 16 个分片，SHARD_ROW_ID_BITS = 6 表示 64 个分片，SHARD_ROW_ID_BITS = 0 就是默认值 1 个分片。

CREATE TABLE 语句示例：`CREATE TABLE t (c int)SHARD_ROW_ID_BITS = 4 PRE_SPLIT_REGIONS=4;`

ALTER TABLE 语句示例：`ALTER TABLE t SHARD_ROW_ID_BITS = 4 PRE_SPLIT_REGIONS=4;`

2. 避免连续自增的主键设计

   对索引组织表来说，它无法利用到 SHARD_ROW_ID_BITS 的优化，可以通过修改序列号的生成方式来构成多个写入分片来分散写入热点。

   AUTO_INCREMENT 主键可通过 AUTO_RANDOM 机制来打散写入热点，详见官网文档。

   例如以下示例为 Snowflake 生成的序列号。经过转换，将最后一位数字移动到左数第二个数字的位置，原左数第二位数字及之后的所有数字向右移动一位。以此来让生成的 ID 跨越 96 MB 的 Region 容量，落在 10 个不同的分片中。左侧为原本要写入表中的序列号值，右侧为转换后的序列号。经过转换后的序列号不再连续且离散程度足以跳过一个 Region 大小，以此方式来分散写入热点：

原始序列号	转换后的序列号
561632371724517376	566163237172451737
561632371728711680	506163237172871168
561632371728711681	516163237172871168
561632371728711682	526163237172871168
561632371732905984	546163237173290598
561632371732905985	556163237173290598
561632371732905986	566163237173290598
561632371732905987	576163237173290598
561632371732905988	586163237173290598
561632371737100288	586163237173710028

表 1. 将连续的写入转换为 10 个分片写入的案例

3. 对单调递增的索引（如时间索引、右增索引等），在插入的时候会有写入热点的问题（原理同表插入数据的过程）。解决办法是通过手工切分热点 Region 的方式来提前打散。切分的办法请参见官网文档《Split Region》章节。

4. 分区表

分区表（partitioned table）可以将一张表的数据分散到多张物理表中，而多张物理表的数据是分散在多个 region 中的，因此通过合理的设计分区规则，可以进一步避免写入热点问题。

8.13.4 4. 读取热点

即使使用了上述的 SHARD_ROW_ID_BITS 方式打散数据在 Region 上的分布，有些小表的热点问题也可能依然存在。典型场景是被频繁读取的参数配置表，高并发的读取集中在

一个或几个 Region 上，有可能成为整笔交易的性能瓶颈。

如果表的数据量比较小，数据存储大概率只涉及到一个 Region ，大量请求对该表进行写入或者读取都会造成该 Region 热点，可以通过拆分 Region 的方式进行调整：

手工拆分：`operator add split-region 1 // 将 region 1 对半拆分成两个 Region`

自动拆分：在 4.0 版本中引入了 Load Base Split 功能，可以实现热点小表的自动 Split。Load Base Split 会基于统计信息自动拆分 Region。通过统计去识别出那些读流量在 10s 内持续超过阈值的 Region，并在合适的位置将这些 Region 拆分。在选择拆分的位置时，会尽可能平衡拆分后两个 Region 的访问量，并尽量避免跨 Region 的访问。在 4.0 版本上已经默认开启该功能。

如果表的数据量比较大，leader 分布不均衡，某些 tikv 节点 Region leader 比较多，不均衡导致的热点需要通过某种机制平衡 leader 分布，平衡方式参考如下：

自动均衡: `curl -G "host:status_port/tables/{db}/{table}/scatter" // 打散相邻 region`

手动均衡: `operator add transfer-leader 1 2 // 把 region 1 的 leader 调度到 store 2`

也可以使用 TiDB 的 Coprocessor Cache 功能，以在 TiDB 实例侧缓存下推给 TiKV 计算的结果，以减少对小表的访问。

推荐做法是在应用侧做缓存处理，使用如 Redis 这样的产品进行缓存。

8.14 分页的最佳实践

8.14.1 1. 分页查询的基本原则

分页查询语句全部都需要带有排序条件，除非业务方明确要求不要使用任何排序来随机展示数据。

详细例子如下：

- 常规分页语句写法（start：起始记录数，page_offset：每页记录数）：

```
sql  select * from table_a t order by gmt_modified desc limit start, page_offset;
```

- 多表 Join 的分页语句，如果过滤条件在某一张表上，通过子查询过滤数据后分页，再 Join：

  低效的写法：

```
select * from table_a t order by gmt_modified desc limit start, page_offset;

select a.column_a, a.column_b .. . b.column_a, b.column_b .. .
```

```
from table_t a, table_b b
where a.xxx.. .
and a.column_c = b.column_d
order by a.yyy
limit start, page_offset;
```

高效的写法：

```
select a.column_a, a.column_b .. . b.column_a, b.column_b .. .
from
(select t.column_a, t.column_b .. .
from table_t t
where t.xxx.. .
order by t.yyy
limit start, page_offerset) a,
table_b b
where a.column_c = b.column_d;

select * from t limit 10000,10;
select * from t order by c limit 10000,10;
```

8.14.2 2. 单字段主键表的分页批处理

常规的分页更新 SQL 一般使用主键或者唯一索引进行排序，再配合 MySQL limit 语法中非常好用的 offset 功能按固定行数拆分页面，然后把页面包装进独立的事务中，从而实现灵活的分页更新。

但是，劣势也很明显：由于需要对主键或者唯一索引进行排序，越靠后的页面参与排序的行数就会越多，尤其当批量处理涉及的数据体量较大时，可能会占用过多计算资源。

首先将数据按照主键排序，然后调用窗口函数 `row_number()` 为每一行数据生成行号，接着调用聚合函数按照设置好的页面大小对行号进行分组，最终计算出每页的最小值和最大值。

```
MySQL [demo]> select min(t.serialno) as start_key, max(t.serialno) as
    ↪ end_key, count(*) as page_size from ( select *, row_number () over (
    ↪ order by serialno) as row_num from tmp_loan ) t group by floor((t.
    ↪ row_num - 1) / 50000) order by start_key;
+-----------+-----------+-----------+
| start_key | end_key | page_size |
+-----------+-----------+-----------+
| 200000000 | 200050001 | 50000 |
| 200050002 | 200100007 | 50000 |
| 200100008 | 200150008 | 50000 |
| 200150009 | 200200013 | 50000 |
| 200200014 | 200250017 | 50000 |
```

```
| ......... | ......... | ......... |

| 201900019 | 201950018 | 50000 |
| 201950019 | 201999003 | 48985 |
+-----------+-----------+-----------+
40 rows in set (1.51 sec)
```

接下来，只需要使用 `serialno between start_key and end_key` 查询每个分片的数据即可。如果修改数据时，也可以借助上面计算好的分片信息，实现高效数据更新。

改进方案由于规避了频繁的数据排序操作造成的性能损耗，显著改善了批量处理的效率。

8.14.3 3. 复合主键表的分页批处理

对于非索引组织表，可以使用隐藏字段 `_tidb_rowid` 做分页使用。

对于索引组织表，可以用如下方式进行分页计算：

制作元信息表

```
mysql> SELECT floor(( t1.row_num - 1 )/ 600000 )+1 rn, min(mvalue),max(
    ↪ mvalue),count(*) FROM (SELECT concat( '(''', customer_id, ''','''',
    ↪ customer_idno, ''')' ) AS mvalue, row_number() over ( ORDER BY
    ↪ customer_id, customer_idno ) AS row_num FROM findpt.customer) t1
    ↪ GROUP BY floor(( t1.row_num - 1 )/ 600000 ) ORDER BY rn;
+--
    ↪ --+-----------------------------------+-----------------------------------
    ↪
| rn | min(mvalue)                       | max(mvalue)                       |
    ↪ count(*) |
+--
    ↪ --+-----------------------------------+-----------------------------------
    ↪
|  1 | ('10000000001','351421198512031871') | ('10000600000','
    ↪ 541420198607276566') | 600000 |
|  2 | ('10000600001','410727197307043818') | ('10001200000','
    ↪ 221518199305165132') | 600000 |
|  3 | ('10001200001','521527198406224414') | ('10001800000','
    ↪ 320209197609305969') | 600000 |
|  4 | ('10001800001','220304197912193073') | ('10002400000','
    ↪ 230504197308067651') | 600000 |
|  5 | ('10002400001','121711197208214015') | ('10003000000','
    ↪ 430112199003258074') | 600000 |
|  6 | ('10003000001','330609198706142725') | ('10003600000','
    ↪ 520519197407128506') | 600000 |
```

```
|  7 | ('10003600001','6211081995081754 76') | ('10004200000','
    ↪ 631319197203254252') | 600000 |
|  8 | ('10004200001','350406198608214809') | ('10004800000','
    ↪ 500827199406068657') | 600000 |
|  9 | ('10004800001','450311198612295355') | ('10005400000','
    ↪ 430713199601229738') | 600000 |
| 10 | ('10005400001','640608199311094703') | ('10006000000','
    ↪ 131222199007068025') | 600000 |
| 11 | ('10006000001','110724197808158121') | ('10006600000','
    ↪ 410909199902088607') | 600000 |
| 12 | ('10006600001','371802199909286692') | ('10007200000','
    ↪ 331616199104157617') | 600000 |
| 13 | ('10007200001','631618198707015770') | ('10007800000','
    ↪ 311424198409271703') | 600000 |
| 14 | ('10007800001','450212199805062337') | ('10008400000','
    ↪ 141520197703176129') | 600000 |
| 15 | ('10008400001','130920197811106553') | ('10009000000','
    ↪ 640206197509055077') | 600000 |
| 16 | ('10009000001','151822197801136758') | ('10009600000','
    ↪ 810620197505228665') | 600000 |
| 17 | ('10009600001','230109198906203721') | ('10010000000','
    ↪ 340408198312036321') | 400000 |
+--
    ↪ --+-----------------------------------+-----------------------------------
    ↪
17 rows in set (26.42 sec)
```

批量处理 SQL 案例

```
delete from customer where (customer_id, customer_idno) >= ('10000000001','
    ↪ 351421198512031871') and (customer_id, customer_idno) <= ('
    ↪ 10000600000','541420198607276566') order by customer_id,customer_idno
    ↪ ;
```

8.15 唯一序列号生成方案

8.15.1 1. 自增列

自增（auto_increment）是大多数兼容 MySQL 协议的 RDBMS 上列的一种属性，通过配置该属性来使数据库为该列的值自动赋值，用户不需要为该列赋值，该列的值随着表内记录增加会自动增长，并确保唯一性。在大多数场景中，自增列被作为无业务涵义的代理主键使用。自增列的局限性在于：自增列只能采用整型字段，所赋的值也只能为整型。假设业务所需要的序列号由字母、数字及其他字符拼接而成，用户是难以通过自增列来获取序列号中所需的数字自增值的。

8.15.2 2. 序列（Sequence）

序列是一种数据库对象，应用程序通过调用某个序列可以产生递增的序列值，应用程序可以灵活的使用这个序列值为一张表或多张表赋值，也可以使用序列值进行更复杂的加工，来实现文本和数字的组合，来赋予代理键以一定的跟踪和分类的意义。从 TiDB v4.0 版本开始提供序列功能，详情请参考官方文档。

8.15.3 3. 类 Snowflake 方案

Snowflake 是 Twitter 提出的分布式 ID 生成方案。目前有多种实现，较流行的是百度的 uid-generator 和美团的 leaf。下面以 uid-generator 为例展开说明。

uid-generator 生成的 64 位 ID 结构如下：

```
| sign | delta seconds | worker node id | sequencs |
|------|---------------|----------------|----------|
| 1bit |    28bits     | 22bits         | 13bits   |
```

- sign：长度固定为 1 位。固定为 0，表示生成的 ID 始终为正数。
- delta seconds：默认 28 位。当前时间，表示为相对于某个预设时间基点 (默认 “2016-05-20”) 的增量值，单位为秒。28 位最多可支持约 8.7 年。
- worker node id：默认 22 位。表示机器 id，通常在应用程序进程启动时从一个集中式的 ID 生成器取得。常见的集中式 ID 生成器是数据库自增列或者 Zookeeper。默认分配策略为用后即弃，进程重启时会重新获取一个新的 worker node id，22 位最多可支持约 420 万次启动。
- sequence：默认 13 位。表示每秒的并发序列，13 位可支持每秒 8192 个并发。

8.15.4 4. 号段分配方案

号段分配方案可以理解为从数据库批量获取自增 ID。本方案需要一张序列号生成表，每行记录表示一个序列对象。表定义示例如下：

字段名	字段类型	字段说明
SEQ_NAME	varchar(128)	序列名称，用来区分不同业务
MAX_ID	bigint(20)	当前序列已被分配出去的最大值
STEP	int(11)	步长，表示每次分配的号段长度

应用程序每次按配置好的步长获取一段序列号，并同时更新数据库以持久化保存当前序列已被分配出去的最大值，然后在应用程序内存中即可完成序列号加工及分配动作。待一段号码耗尽之后，应用程序才会去获取新的号段，这样就有效降低了数据库写入压力。实际使用过程中，还可以适度调节步长以控制数据库记录的更新频度。

最后，需要注意的是，上述两种方案生成的 ID 都不够随机，不适合直接作为 TiDB 表的主键。实际使用过程中可以对生成的 ID 进行位反转（bit-reverse）后得到一个较为随机

的新 ID。

8.16 流程规范

流程规范主要针对线上客户生产集群的调试，临时统计，上线，变更等操作流程规范定义，避免测试或者其它误操作情况造成集群性系列影响。

1. 所有的建表操作需要提前告知 DBA 该表涉及的查询 SQL；
2. 所有的建表需要确定建立哪些索引后才可以建表上线；
3. 所有的改表结构、加索引操作都需要将涉及到所改表的查询 SQL 发出来告知 DBA 等相关人员；
4. 在建新表加字段之前，建议开发人员提前发出给 DBA 评估、优化和审核；
5. 批量导入、导出数据必须提前通知 DBA 协助观察；
6. 大批量统计更新，如临时统计，避开高峰期，并通知 DBA；
7. 推广活动或上线新功能必须提前通知 DBA 进行流量评估；
8. 及时处理已下线业务的 SQL。

©2019 PingCAP All Rights Reversed.